



TIME-SERIES PREDICTION AND ELMAN NETWORK

INT301 Bio-computation, Week 9, 2025



Time Series

- 按时间排序
- 相等时间间隔
- 可比较的观测值

■ What is a time series?

– Time Series is a series of timely ordered, comparable observations y_t recorded in equidistant time intervals

■ Examples:

– Stock prices 股票

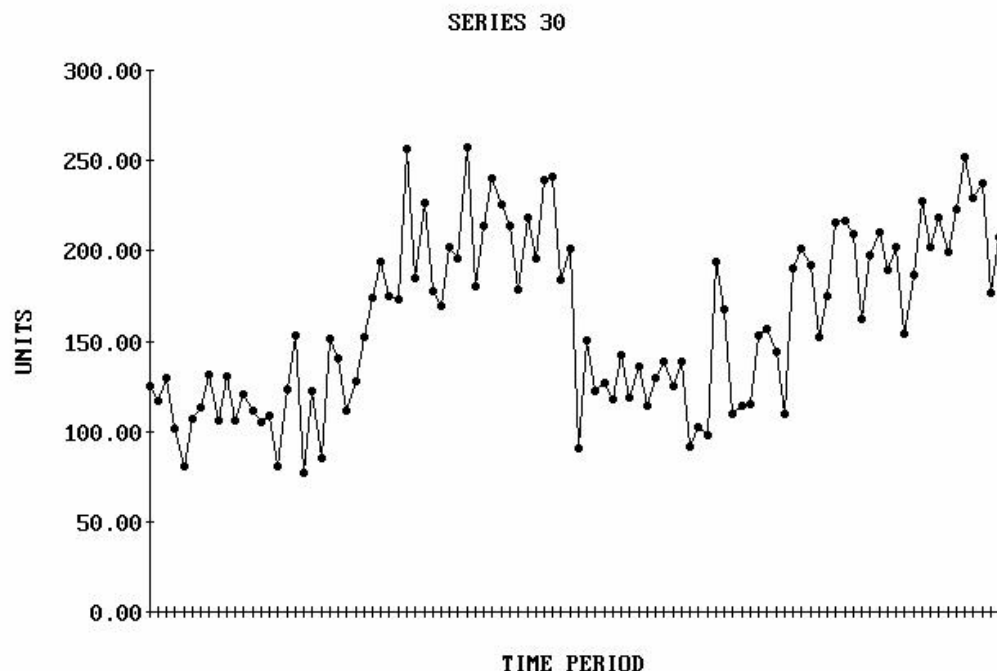
– Temperature readings 温度变化

Y_t represents the t^{th} period observation, $t=1,2 \dots n$

y_t 代表第 t 个时刻观察到的数据

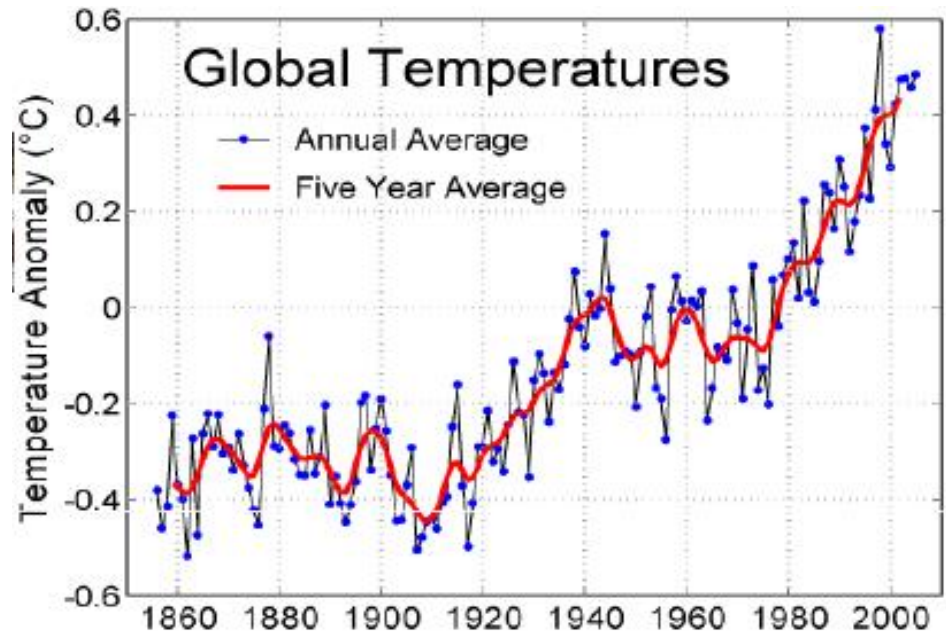
x 代表时刻变化

时间序列是指一系列按时间排序的、可比较的观测值 y_t ，这些观测值是在相等的时间间隔下记录的。

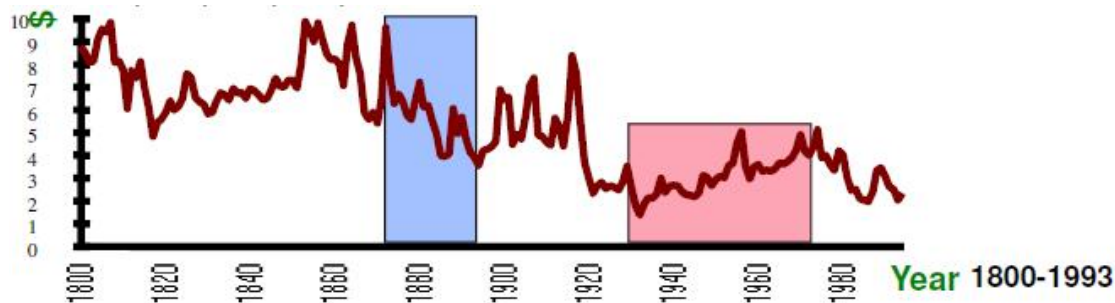


Examples of Time Series

Global Warming



Real Copper Prices in \$ per KG



Electroencephalography (EEG)



Time Series Models

数据与自己之前的结果有关联, 称为“自关联” Autocorrelation

■ Modeling of a time series

一个时间序列中, 连续时间点上的值通常是相关的。否则, 预测是不可能的。

- Values of a time series at successive time indices are often correlated. Otherwise, prediction is impossible.
- The statistical properties do not change with respect to time 并且数据整体上所表现出的规律必须是相同的

这个序列的统计特性不会随着时间而改变。

宽义平稳
(WSS) 随机过程

■ WSS (wide-sense stationarity) random process

- The mean and autocovariance do not vary with respect to time
- Variance is finite for all times

1. “均值和自协方差不随时间变化”

均值恒定: 序列的“平均水平线”是平的, 不会随着时间向上或向下“漂移” (比如一个持续上涨的股价序列, 就不是平稳的)。

自协方差恒定: “今天”和“昨天”的关联方式 (自协方差), 与“10 年前的那一天”和“10 年前的前一天”的关联方式是一样的。这保证了“规律”的稳定性。

2. “方差在所有时间都是有限的”

這是一個技術條件, 意思是數據的波動不會“爆炸”到無限大。

未知的动态（过程）” (Unknown dynamics (process)) P: P 代表了产生我们数据的“真实世界规则”，比如：P 可能代表“（一个城市里）所有复杂的、影响天气变化的物理规律”；或者“（一个经济体中）所有影响股票市场的复杂人类行为和经济法则”。P 是我们永远无法完全知道的。

“观测到的时间序列” (Observed time-series):

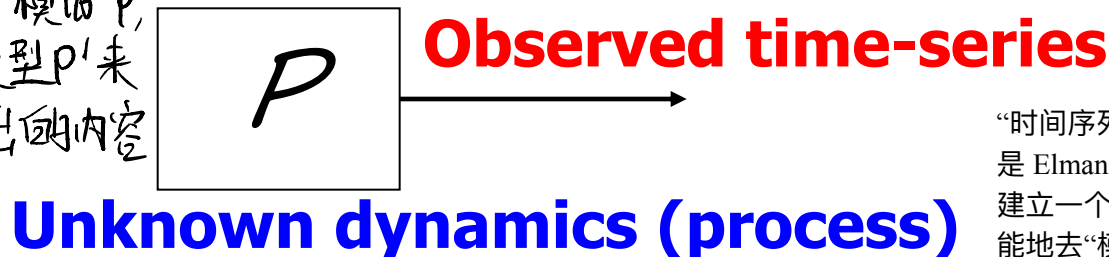
这是 P 这个“黑匣子”吐出的“输出”。比如：我们观测到的“温度读数”或“股票价格”。这个输出就是我们手头的数据，即 $\{x(t_0), x(t_1), \dots, \}$

- In statistics and machine learning, a time-series is often described by a sequence of vectors (or scalars) which depend on time t:

$$\{x(t_0), x(t_1), \dots, x(t_{i-1}), x(t_i), x(t_{i+1}), \dots\}$$

It's the output of some process P that we are interested in:

如果我们能成功地“模仿”P，那么我们就用模型P'来预测P在未来要输出的内容



“时间序列建模”（无论是用统计学还是 Elman 网络）的最终目的，就是建立一个“模型”P'，让P'尽最大可能地去“模仿”那个“未知的真实过程”P。

连续信号 $x(t)$: 这代表了那个“真实过程 P”在每一毫秒、每一纳秒都在连续不断地产生输出。比如“真实的气温”是一条平滑的、连续的曲线。但是我们不可能记录下那个连续的“真实信号”。我们只能“每隔一段时间”（比如每小时）去“看一眼”（即“采样”）。这就是所谓的“时间间隔”。例如，如果这个间隔是 Δt ，然后单位是 1 小时，那么我们得到的数据就是： $\{x(0), x(1\text{小时}), x(2\text{小时}), \dots\}$

Time Series Models

- The continuous signal $x(t)$ is sampled at discrete points to get a series
- In uniform sampling, if sampling period is Δt

$$\{x[t]\} = \{x(0), x(\Delta t), x(2\Delta t), x(3\Delta t), \dots\}$$

从当前时间 t 向后看, 我们拥有一系列历史数据 $\{x[t], x[t-1], \dots\}$

Problem of Predicting the Future

Extending backward from time t , we have time series $\{x[t], x[t-1], \dots\}$. From this, we now want to estimate x at some future time 接下来预测未来的值

$\hat{x}$: 预测
 s : 要预测多少时间之后
 $\hat{x}[t+s] = f(x[t], x[t-1], \dots)$ 对未来的预测值
RBF问题: $y = f(x_1, x_2, \dots)$
这里的 x 是不同的特征
例如 房价 $f = (\text{面积}, \text{地段})$

s is called the **horizon of prediction**. At beginning, let's predict just one time sample into the future, $\Rightarrow s = 1$.

This is a function approximation problem. Time series 问题: $\hat{x}[t+1] = f(x[t], x[t-1], \dots)$

Here's how we'll solve it:

1. Assume a **generative model**. 两者都是找到一个 f , RBF 关注输入特征, Time series 类
2. For every point $x[t_i]$ in the past, train the generative model 逐时间点 with what preceded t_i as the inputs and what followed t_i as the desired. 明天的价格 = 今天的价格 和 昨天的价格
3. Now run the model to predict $\hat{x}[t+s]$ from $(x[t], x[t-1], \dots)$

模型 f 的通用训练策略:

1. 假设一个模型: 假设存在一个模型 f (比如 Elman 网络) 能够胜任函数逼近任务。

2. 训练模型: 使用滑动窗口 (sliding window) 的方法, 对每一个点 $x[t+1]$, 用它之前的数据作为输入, 之后的数据作为目标, 来训练这个模型

例如有一组数据: $[10, 20, 30, 40, 50, \dots]$, 训练样本1, $[10, 20]$, 期望得到 30,
训练样本2, $[20, 30]$, 期望得到 40, ...
最终得到规律 $f(a, b) = a + b$.

3. 运行模型: 用模型 f 学习到的规律来预测 $\hat{x}[t+1]$.

比如想预测 $\hat{x}[t+1]$, 那就用 $x[t]$ 和 $x[t-1]$ 作为参数 再用先前学习到的规律。

TS 重要性:

防患于未然 (Prevention):

如果能预测到坏事 (比如机器故障、金融危机) 要发生, 我们就能识别出先兆, 提前采取纠正措施来避免它发生。

减损 (Mitigation):

对于那些不可避免的坏事 (比如台风、特定经济周期), 预测能让我们提前做好准备, 从而减轻它们带来的冲击和损失。

获利 (Profiting):

这是最直接的动力。在金融市场 (股票、汇率) 中, 如果你能比别人更准地预测未来价格, 你就能赚钱。

TS 预测困难的原因:

数据有限 (Limited quantity of data):

有时候历史数据太短了, 不够用来训练复杂的模型 (无法很好地划分训练集和测试集)。

噪声 (Noise):

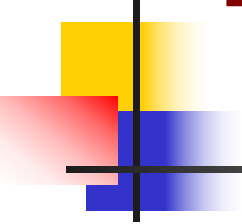
数据里往往混杂着错误的、随机的波动。模型很容易被这些噪声“带偏”, 学到错误的规律。

非平稳性 (Non-stationarity):

这是最大的敌人。正如我们之前所说, 如果“基本面” (规则) 随着时间改变了 (比如市场结构变了), 那么基于历史数据训练的模型就会失效。

方法选择困难:

到底是用传统的统计学方法 (如 ARIMA), 还是用人工智能 (神经网络)? 这往往需要权衡。



Time Series Prediction

- Time series examples
 - Financial (e.g. stocks, exchange rates)
 - Physically observed (e.g. weather, river flow)
- Why is it important?
 - Preventing undesirable events by forecasting the event, identifying the circumstances preceding the event, and taking corrective action so the event can be avoided;
 - Forecasting undesirable, yet unavoidable, events to preemptively lessen their impact;
 - Profiting from forecasting (e.g., financial markets)

Time Series Prediction

- Why is it difficult?
 - Limited quantity of data
 - observed data series sometimes too short to partition
 - Noise
 - erroneous data points
 - Non-stationarity
 - fundamentals change over time, nonstationary
 - Forecasting method selection
 - statistics, artificial intelligence
- Neural networks have been widely used as time series forecasters: most often these are feed-forward networks which employ a **sliding window** over the input sequence.
- The neural network sees the time series $X_1, \dots, X_n$ in the form of many mappings of an input vector to an output value.

通过这种方式，神经网络不需要理解“时间”，它只需要学习这种固定的“输入向量 -> 输出值”的映射关系即可。

在输入序列上使用一个“滑动窗口”

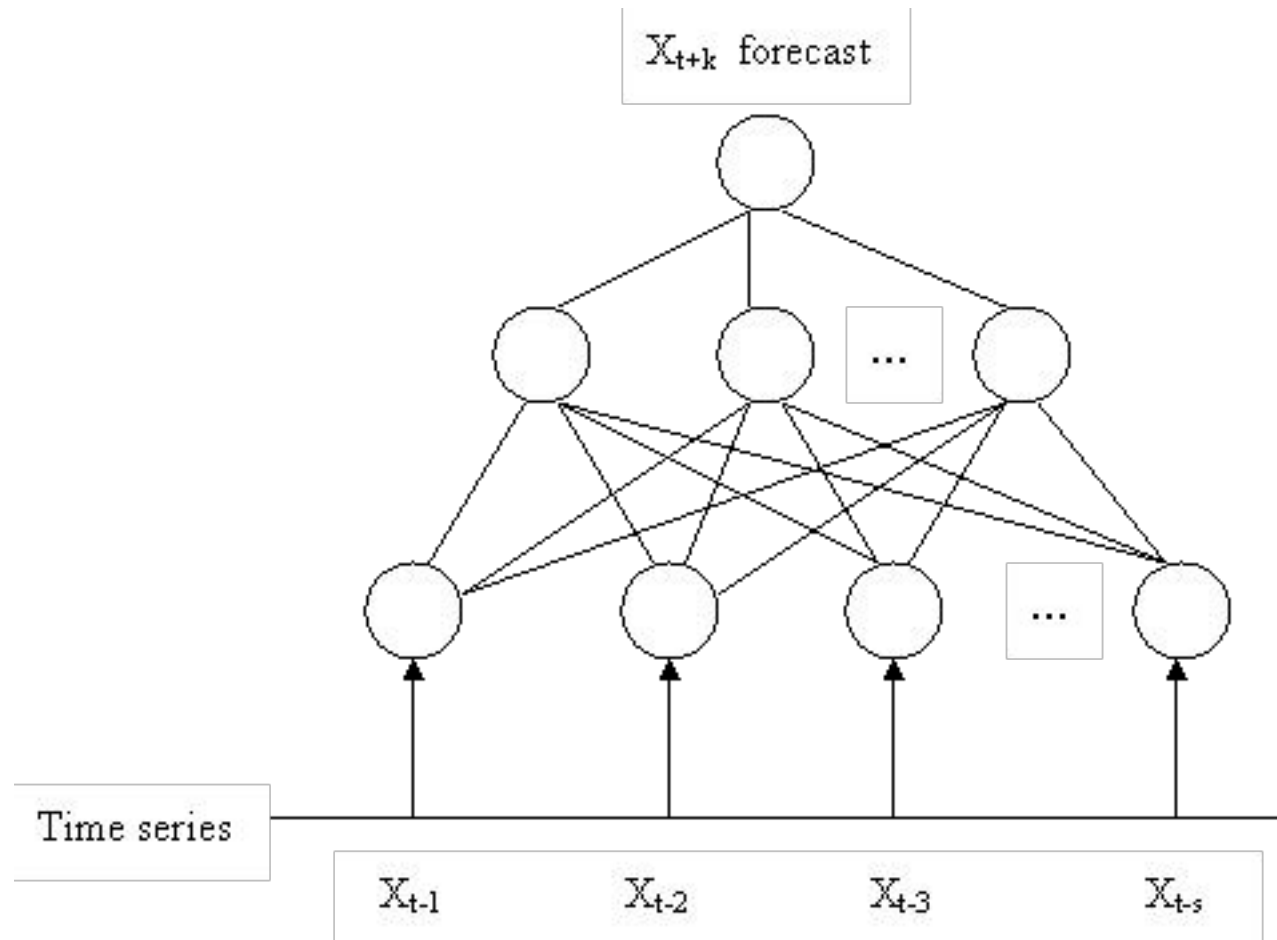
从左到右依次进行训练

Time Series Prediction with ANN



- A number of adjoining data points of the time series (the input window $X_{t-s}, X_{t-s+1}, \dots, X_t$) are mapped to the interval $[0,1]$ and used as activations for the input of the input layer.
- The size **s** of the input window corresponds to the number of input units of the neural network.
- In the forward path, the activations are propagated over hidden layers to output units. The error used for the BP training is computed by comparing the value of the output unit with the value of the time series at time $t+1$.
- Training a MLP network with BP learning algorithm usually requires that all representations of the input set (called one *epoch*) are presented many times.

Time Series Prediction with ANN



输入层神经元接收的是 $X_{t-1}, X_{t-2}, \dots, X_{t-s}$ ，这些是过去的数据， s 是窗口大小。

对于输入的数据，需要进行“归一化”。神经网络通常不喜欢处理差异值巨大的数据，把它们压缩到 0 到 1 之间有助于网络更快速收敛。

输入单元数 = 窗口大小 s 。这个结构必须是固定的。

前向传播：架构图顶部：输出层只有一个节点，输出的是 $X_{t+k} \text{ forecast}$ ；通过 $k=1$ ，也就是紧接着下一时刻。

前向路径：信号从输入层 $\rightarrow$ 隐藏层 $\rightarrow$ 输出层传播。

误差计算：网络算出一个预测值，然后把它与真实发生的 $t+1$ 时刻时刻比较，计算误差。

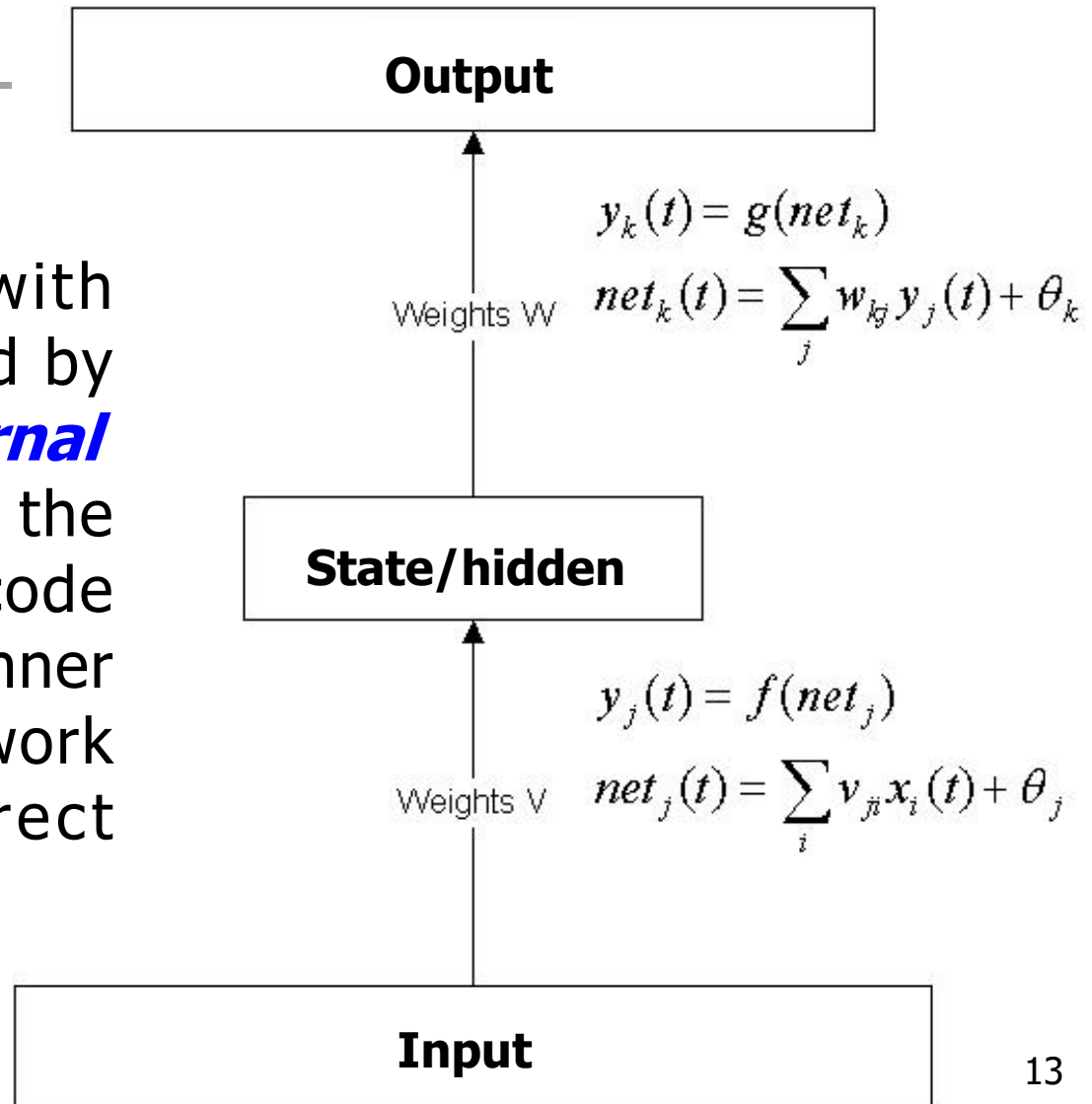
BP 反向传播，RBF 训练是一次性的 (k -Means + 伪造)。这里 MLP 网络使用迭代式梯度下降。

Epoch 轮次：要让网络反复地看这一堆历史数据，每一遍叫一个 epoch。网络在每一遍中通过 BP 一点点修正，直到误差足够小。

前馈神经网络：没有记忆，无法处理上下文信息。

Revisit of feed forward NN

A feed forward NN with hidden nodes, trained by e.g. BP, develops *internal representations* for the input signals that recode those signals in a manner that enables the network to produce the correct output.



MLP和RBF这种无记忆,先天不足.(静态网络)

Motivation for dynamic networks

- Sometimes, we require our model to be sensitive to inputs that were presented some time ago.
- In other words, our requirement is not met by a function (no matter how complex it may be) and demands **a model which maintains a *state*, or *memory***, over several presentations.
- Which applications need memory?
 - **sequential input** (e.g. language understanding, robot exploration, ...),
 - **sequential output** (e.g. speech, route planning, ...), or combinations of the above.

简单的 $f(x) \rightarrow y$ 映射是不够的。

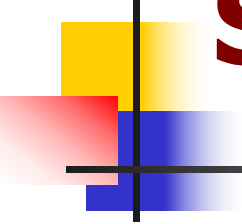
我们需要一个能够“维持”一个“状态” (state) 或“记忆” (memory) 的模型。

静态网络：单个输入信号 $\rightarrow$ 单个输出信号

动态网络：它们学习的是涉及“序列” (sequence) 的映射。比如：输入 (一个信号) $\rightarrow$ 输出 (一个序列)；或者更常见的是：输入 (一个序列) $\rightarrow$ 输出 (一个序列)。

Sequence Learning

- MLP & RBF networks are **static networks**, *i.e.*, they learn a mapping from a single input signal to a single output response for an arbitrary large number of pairs.
- **Dynamic networks** learn a mapping from a single input signal to a sequence of response signals, for an arbitrary number of pairs (signal, sequence).
- Typically the input signal to a dynamic network is an element of the sequence and then the network produces the rest of the sequence as a response.
- **To learn sequences we need to include some form of memory (short term memory) to the network.**



Sequence Learning

- We can *introduce* **memory effects** with two principal ways:
 - **Implicit:** e.g., Time lagged signal as input to a static network or as recurrent connections
 - **Explicit:** e.g., Temporal Backpropagation Method
- In the implicit form, we assume that the environment from which we collect examples of (input signal, output sequence) is *stationary*. For the explicit form the environment could be *non-stationary*, i.e. the network can track the changes in the structure of the signal.

隐式记忆 (Implicit)

方式 1: "Time lagged signal as input to a static network"

(“将‘时延信号’作为‘静态网络’的输入”)

这正是“滑动窗口 MLP”！我们的输入是 $[x_{t-1}, x_{t-2}, \dots, x_{t-s}]$ 。这个“输入向量”本身就是一种“记忆”，它把“过去 s 步”的历史“显式地”编码（“滞后”）并“喂”给了一个本身没有记忆的“静态网络”。

方式 2: "as recurrent connections"

(“作为‘循环连接’”)

这就是 Elman 网络。我们不再通过一个巨大的“输入向量”来“喂”给网络历史，而是给网络一个“内部循环”。这个“循环连接”允许网络将上一个时间点的“状态”（它的“记忆”）反馈给自己，用来和当前时间点的“新输入”相结合。

显式记忆 (Explicit)

这是一个更复杂的概念（如“时间反向传播”），它允许网络“回顾”更久远的历史。但目前，“隐式记忆”的“循环连接”是我们关注的重点。

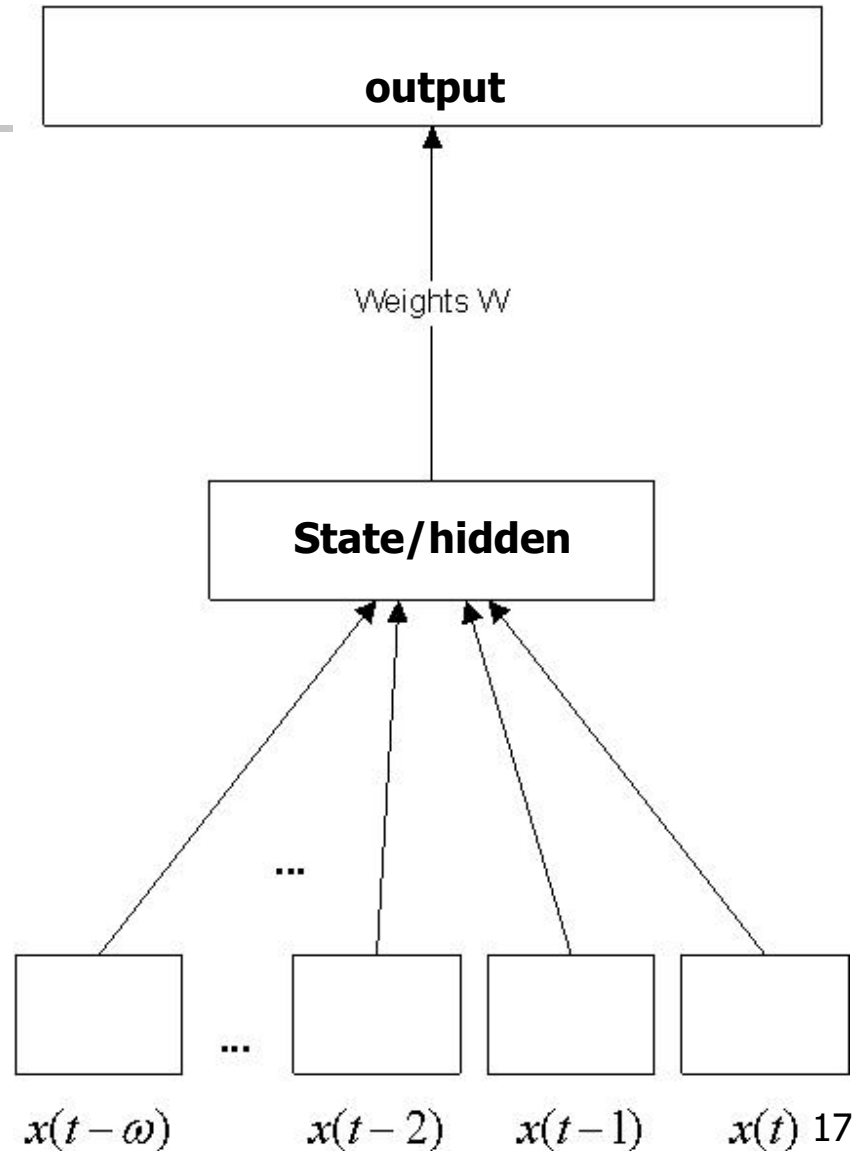
最后的关键点 (平稳性):

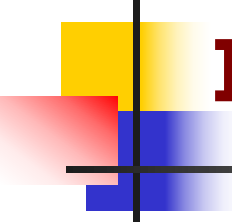
“隐式”方法（比如“滑动窗口”）通常假设环境是“平稳的” (stationary)，即 f 这个规则是固定不变的。

“显式”方法（或更强大的循环网络）可以跟踪“非平稳” (non-stationary) 的信号，因为网络的“状态/记忆”本身可以随着时间而演变和适应。

Time Delayed Networks

- Signals can be **buffered externally** and presented at **additional input nodes** (input banks).
- **This is called**
Time Delayed Networks
- Time Delayed Networks illustrate the implicit representation approach for sequence learning, which combine a static network (e.g. MLP / RBF) with a memory structure.





Dynamical Neural Networks

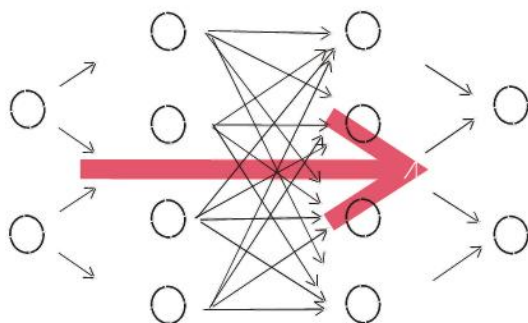
Introducing time

- **Recurrent networks** can explicitly deal with inputs that are presented *sequentially*, as they would almost always be in real problems.
- Recurrent networks are fundamentally different from feed forward networks in that they **not only operate upon an input but also a state**.
- The ability of the net to reverberate and sustain activity can serve as a **working memory**.
- Such nets are **Dynamical Neural Networks**.

From feed-forward to recurrent NN

在 t 时刻，隐藏层计算出一个“激活值”（即它的“思考结果”）。

因为有这么个“循环”，这个“思考结果”不会在 $t+1$ 时刻消失，而是会被“拷贝”并“反馈”回隐藏层，作为 $t+1$ 时刻的“额外输入”。



- Connections only "from left to right", no connection cycle
- Activation is fed forward from input to output through "hidden layers"
- No memory



- At least one connection cycle
- Activation can "reverberate", 回响
- System with memory
- Mathematically, RNNs implement dynamical systems.

Elman Network

- Elman neural network was proposed by Jeffrey L. Elman, 1990.
- It is a simple recurrent neural network
- Elman network has a powerful prediction capability



Elman 网络的核心思想是，它的输出不仅取决于“当前的输入”，还取决于一个“内部状态”（即“记忆”），这个“内部状态”会随时间不断更新。

Elman Network

- 状态-输出映射 (产生预测) $y = F_w(z^t)$

y 是网络在当前时刻 t 的输出
网络输出只取决于它当前的状态 z^t

- Elman network which typically distinguishes between a **state-output function** and an **input-state mapping**.

$$y = F_w(z)$$

$$z^t = F_v(z^{t-1}, x)$$

where y is the output, x is the input and z is the state. t is an index in time.

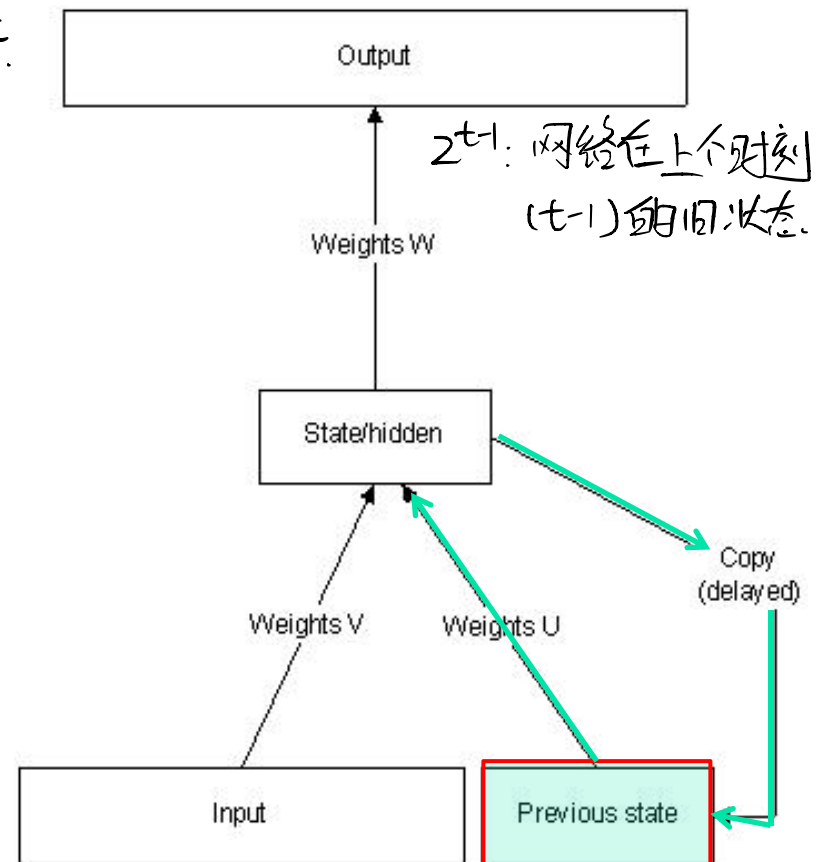
- 输入-状态映射 (更新“记忆”) :

$$z^t = F_v(z^{t-1}, x)$$

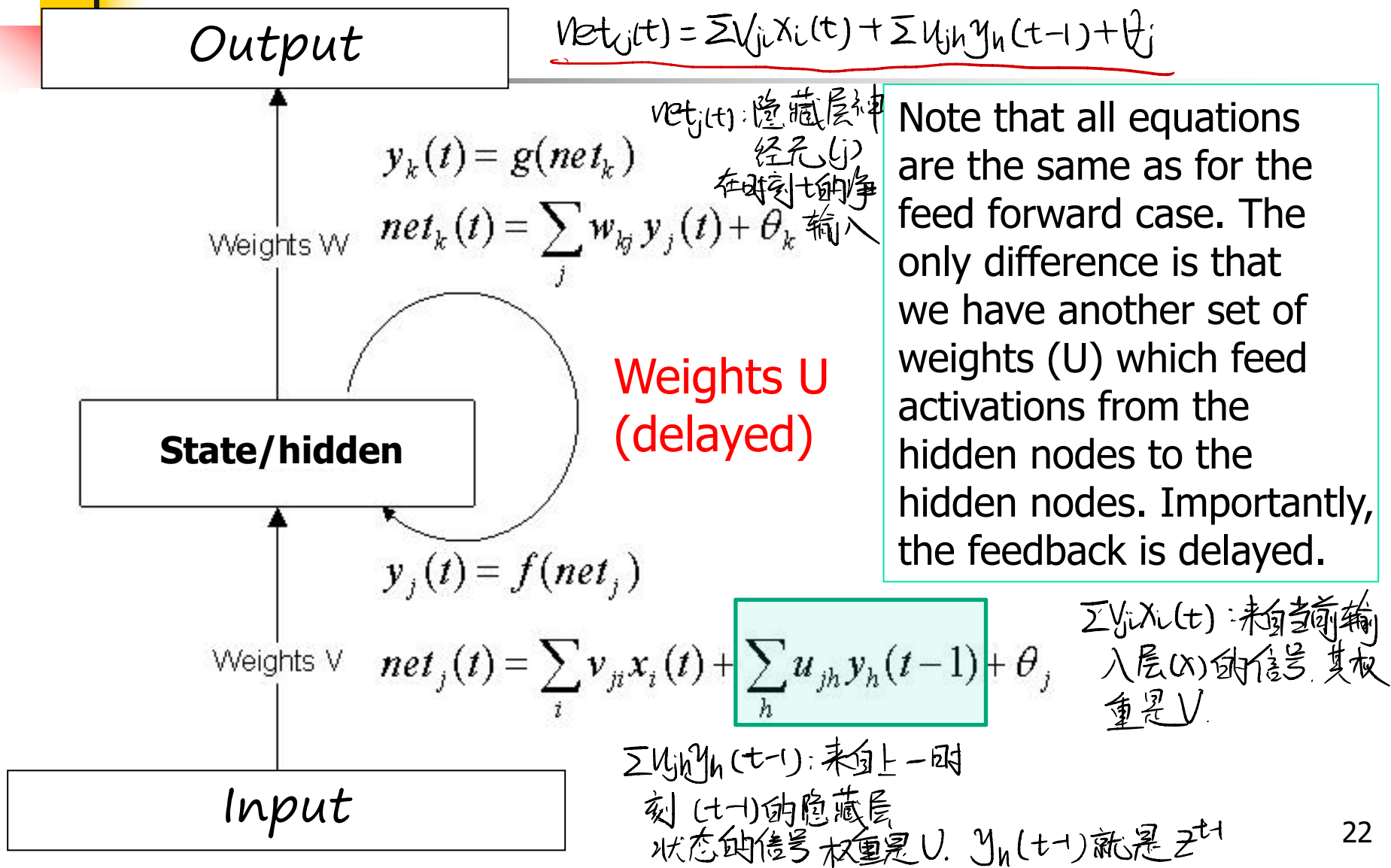
- z^t 是网络在当前时刻 t 的状态 (state) 或记忆

- 这个当前状态是两个东西的函数

x : 当前的输入信号



In Elman net, the internal representations are now engaged in the next step as an additional input.



Elman网络有一个额外的权重矩阵 U (循环权重) 专门用来学习记忆 (z^{t-1}) 对当前状态 z^t 的影响。(普通的MLP只有 V , 没有 U)

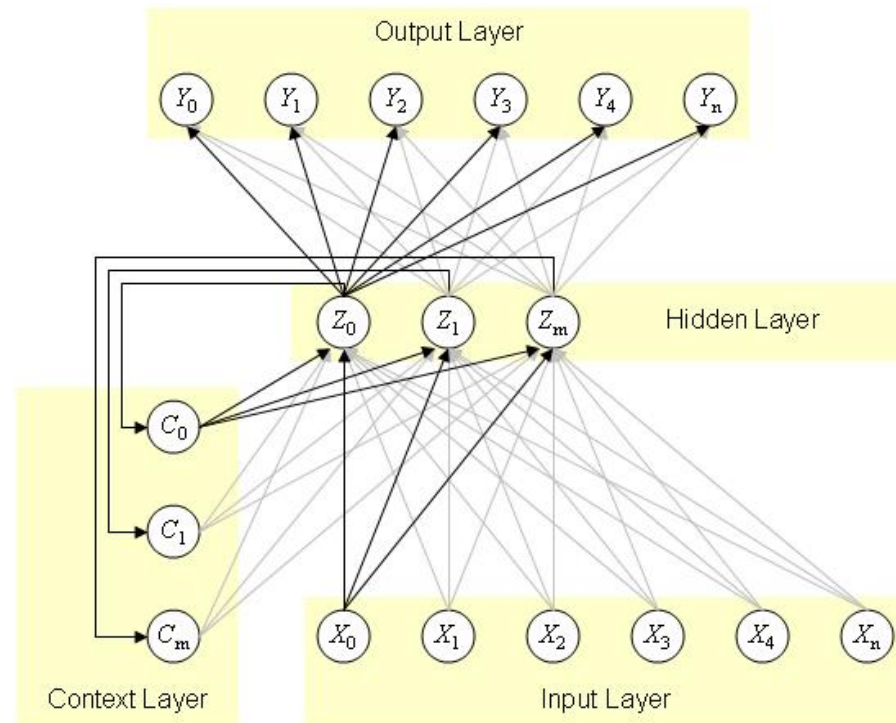
Elman Network

Four layers of Elman network

- **Input layer**
- **Hidden layer** forming internal representation
- **Context layer** 记忆实现的地方.

The value of the context neuron is used as an extra input signal for all the neurons in the hidden layer one time step later. In an Elman network, the weights from the hidden layer to the context layer are set to one and are fixed because the values of the context neurons have to be copied exactly

- **Output layer** linear combination



“上下文层”是如何工作的？（这就是那个循环）

- （看图中的箭头）
- 步骤 1：在时刻 t ，输入层 (X) 和上下文层 (C) 一起作为输入，被送入隐藏层 (Z)。
 - （上下文层 C 中存储的是上一刻 $t - 1$ 的隐藏层状态 z^{t-1} ）。
- 步骤 2：隐藏层 (Z) 结合了**“新输入 X ”和“旧记忆 C ”，计算出了“新状态 z^t ”**。
- 步骤 3：这个“新状态 z^t ”被同时发送到两个地方：
 - (a) 向上 $\rightarrow$ 发送给输出层 (Y)，用于计算当前的预测 y^t 。
 - (b) 向下（循环） $\rightarrow$ “拷贝”到上下文层 (C) 中。
- 步骤 4：上下文层 (C) “保持”住这个状态 z^t ，等待下一个时刻 $t + 1$ 的到来，届时它将作为“旧记忆”再次被送入隐藏层。
- 注意：幻灯片提到，从“隐藏层”到“上下文层”的权重是固定的 1，因为这一步只是一个**纯粹的“拷贝”**动作。

如何找到 U, V, W 这些权重

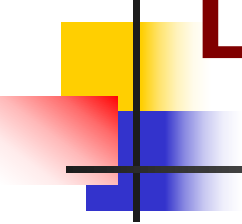
RBF网络训练有两个阶段（k-Means + 伪逆法 $\overline{V}^+$ ）一次性的数学求解。

Elman网络：不能用伪逆法， V 矩阵的循环特性，隐藏层输出 z^t 不仅是 x 的函数，还是 z^{t-1} 的函数。

而 z^{t-1} 又是 x^{t-1} 和 z^{t-2} 的函数。深度嵌套的非线性关系。

Elman 采用反向传播算法，与 MLP 与 RBF 一样，都是为了最小化误差平方和 (SSE)。

$$E = \sum [y(k) - d(k)]^2$$



Learning algorithm for Elman Network

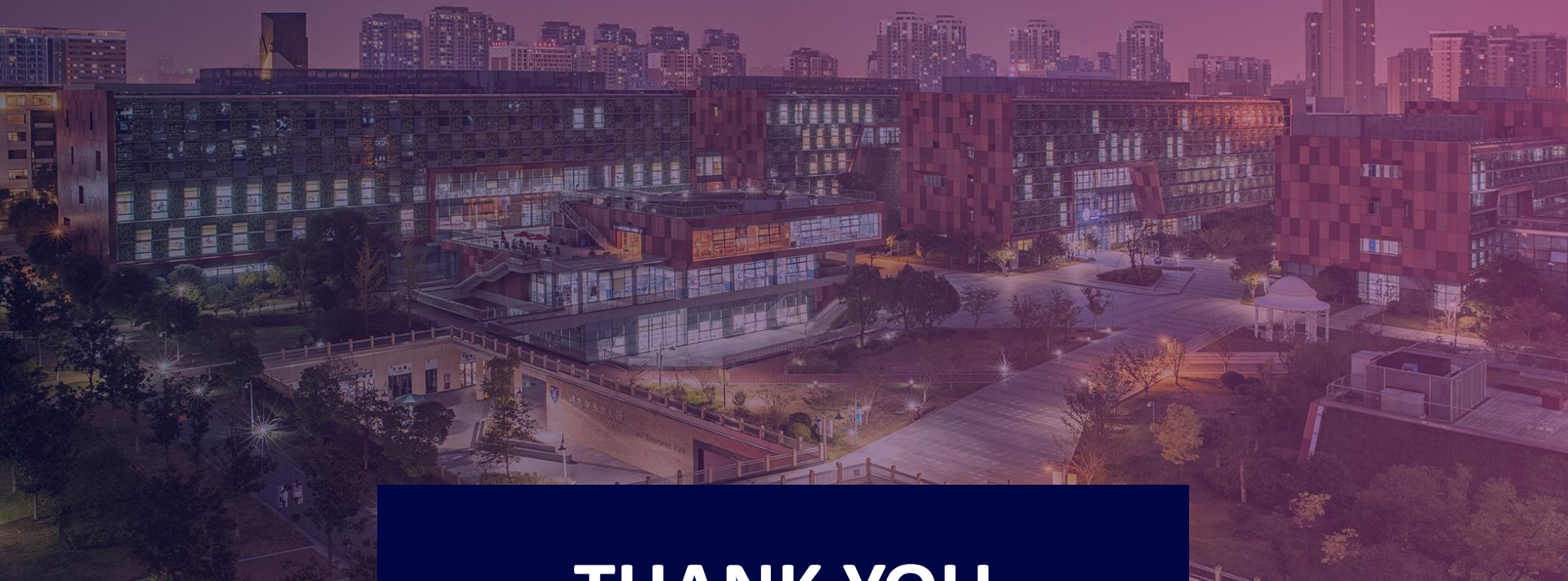
- Elman network adopts BP algorithm

- The error function:

$$E = \sum_{k=1}^n [y(k) - d(k)]^2$$

当误差从输出层反向传播时，它不仅会向下传播到隐藏层（以更新 W 和 V ），还会沿时间轴向后（沿着上下文层）传播到上个时刻（更新 U ）。

where $d(k)$ is the expected output (target)



THANK YOU



VISIT US

WWW.XJTLU.EDU.CN



FOLLOW US

@XJTLU



Xi'an Jiaotong-Liverpool University
西交利物浦大學